

AnchorSlice: Anchor-Guided Static Slicing for Token-Efficient Smart Contract Vulnerability Detection

SAJAD AGHANASIRI¹, SINA ZAKER¹, ALIREZA SHAMELI-SENDI¹

¹Faculty of Computer Science and Engineering, Shahid Beheshti University (SBU), Tehran, Iran (e-mail: {s.aghanasiri, a_shameli}@sbu.ac.ir)

Corresponding author: Alireza Shameli-Sendi (e-mail: a_shameli@sbu.ac.ir).

This work was not supported by any specific grant but was conducted as part of academic research at Shahid Beheshti University.

ABSTRACT Automated vulnerability detection is a prerequisite for deploying smart contracts safely, because a deployed contract is immutable and every defect in it remains reachable by an adversary. Detectors that reason directly over source code have become increasingly capable, but their cost, latency and context capacity are bounded by the number of tokens they read, and a typical Solidity contract is dominated by comments, library code and boilerplate that cannot host a vulnerability. This paper presents AnchorSlice, a static, model-free stage that makes smart contract vulnerability detection token-efficient. From one anchor set per category of the DASP taxonomy it builds two components: a slicer that reduces each contract, before it reaches the detector, to the fragments capable of hosting each category, and a set of anchor-derived decision rules that tell the detector what each category means in the corpus at hand. AnchorSlice is purely lexical: category-specific anchor expressions and brace matching decide what survives, overlapping regions are merged once and tagged with every category they may host, and unmodified library constructs are collapsed to stubs under a rescue rule that preserves evidence. The slicer needs no compilation and processes a contract in 7.6 ms on average. On 200 contracts from the DIVE benchmark it retains 56.5% of source characters while preserving evidence for all 463 labelled category instances, a retention of 1.00 in every category. Three arms over the same 200 contracts separate what each component contributes. Against the way such a detector is used in practice, which is to submit the complete contract with a one-line instruction per category, the decision rules raise the F1-score from 0.342 to 0.695 and cut missed vulnerabilities from 314 to 92 (McNemar exact test over 1,600 paired cells, $p \approx 7.1 \times 10^{-24}$). Adding the slicer holds that quality, 0.693 against 0.695 with no detectable difference ($p = 0.57$), while reducing token consumption by 62.0%, cost by 63.2% and wall-clock time by 68.8%, to 4,689 tokens, \$0.032 and 8.4 seconds per contract. Against ordinary practice the method therefore doubles the F1-score while spending 55.2% fewer tokens, and the result is unchanged on the 130 contracts that played no part in its development. One static artefact makes the same detector both better and cheaper.

INDEX TERMS blockchain security, smart contract vulnerability detection, static slicing, token efficiency

I. INTRODUCTION

DETECTING vulnerabilities in smart contracts before they are deployed is one of the few effective defences available on public blockchains, and detectors that read contract source code directly have become considerably more capable in recent years. Their cost, however, grows with the length of what they read, while much of a deployed contract is irrelevant to any vulnerability. This section motivates the problem of *token-efficient smart contract vulnerability detection*. It first explains why contract defects are costly and why existing analysers leave the detection problem open,

then shows how the input length of source-level detectors has become a limiting factor, and finally introduces AnchorSlice and summarises the contributions of this work.

Smart contracts turn contractual clauses into code that executes without a trusted intermediary [1], and on Ethereum they hold and move assets directly [2]. That property is also the problem. A contract is immutable once deployed, its bytecode is public, and any defect in it is permanently reachable by an adversary who can read the source. The 2016 DAO incident, in which a reentrancy fault drained roughly sixty million dollars, made this concrete and motivated the

first generation of automated analysers [3]. Practitioners subsequently organised the recurring fault patterns into the Decentralized Application Security Project (DASP) Top 10 [4], which remains the vocabulary against which most detection work is evaluated.

A decade of tooling has not settled the problem. When nine widely used analysers were run over 47,587 deployed contracts, they collectively identified only a minority of the annotated faults and flagged 97% of the corpus as vulnerable, a false-positive rate that makes unattended use impractical [5]. Controlled bug injection reaches the same conclusion from the opposite direction: injected faults of well-understood shapes go undetected by tools designed specifically to find them [6]. The difficulty is not that analysers such as Slither [7] reason poorly about control flow, but that many real defects are semantic. Whether a missing modifier matters depends on what the function does, and no fixed data-flow pattern captures that. Learning-based detectors were introduced partly to close this gap [8].

Detectors that reason over source code with pre-trained language models changed the trade-off again. Because such models carry general knowledge of programming idioms and security reasoning, a contract can be examined directly, without a hand-written rule for each fault class. Pipelines that pair model reasoning with static confirmation [9], that split auditing into generation and criticism [10], or that fine-tune a detector and then argue about its output through cooperating agents [11] all report detection quality beyond that of pattern-based tools, and a recent systematic review counts a rapidly growing body of such work [12]. In all of these systems the language model is a component of the detector rather than the object of study; what they share is that the detector consumes the contract source.

This dependence has a cost that scales with input length. A source-level detector is billed per token and bounded by a context window, and long inputs also degrade reasoning: accuracy falls when the relevant evidence sits in the middle of a long input rather than near its edges [13]. For smart contracts the waste is easy to see. A fixed-supply ERC-20 token is several hundred lines of library boilerplate wrapped around perhaps thirty lines that could plausibly hold a flaw, yet the whole file is submitted. Prompt compression attacks the general version of this problem by using a small model to drop low-information tokens [14], but that approach is domain-agnostic, itself consumes model capacity, and offers no guarantee that security-relevant code survives.

We take the opposite route and exploit the fact that vulnerability classes have lexical footprints. A reentrancy fault requires an external call, a bad-randomness fault requires a block-derived value, and an arithmetic fault requires an operator. Code containing none of these cannot host the corresponding defect, and deciding that requires no model. This paper introduces AnchorSlice, a purely static slicing stage that keeps, for each DASP category, only the regions whose tokens make the category possible, and hands the detector a compact annotated slice instead of the full file.

The design borrows the intuition of program slicing [15] but deliberately stops short of dependence analysis, so that the slicer runs in milliseconds and never needs to compile. It extends the candidate-pruning idea of our earlier LineGuard framework [16], which restricts the detector’s view for one vulnerability category at a time, to all eight categories in a single pass with a measurable evidence guarantee. We evaluate AnchorSlice on the DIVE multi-label benchmark [17], whose 22,330 contracts are annotated for the eight DASP categories we target.

The primary contributions of this paper are summarised as follows:

- We present *AnchorSlice*, a detector-agnostic, purely lexical method for token-efficient smart contract vulnerability detection, composed of two static components built from one anchor set for DASP categories 1–8: a slicer, with a visibility gate, a tag-preserving merge that emits each source region once while labelling it with every category it may host, and a library-collapse step with a rescue rule that restores lost evidence; and a set of anchor-derived decision rules that fix what each category means for the detector.
- We validate the slicer against the DIVE benchmark and show that it retains evidence for all 463 labelled category instances across 200 contracts, a retention of 1.00 in every category, while emitting 56.5% of the original characters at an average of 7.6 ms per contract. We also report which semantic restrictions are compatible with the benchmark’s annotation policy and which are not.
- We report a paired evaluation over three arms that credits each component separately: the decision rules raise the F1-score from 0.342 to 0.695 on complete sources, and the slicer then cuts token consumption by 62.0%, cost by 63.2% and latency by 68.8% with no detectable change in detection quality. We test the significance of each step and analyse which anchors carried information and which misled the detector.

Taken together, these contributions ask how far a static, model-free stage can go: how much of a contract can be removed while every labelled vulnerability keeps its evidence, what that removal is worth in tokens, cost and latency, and what happens to detection quality when the same detector is driven this way rather than in the usual one.

The remainder of this paper is structured as follows. Section II reviews previous work on smart contract vulnerability detection and input reduction. Section III introduces AnchorSlice and its design. Section IV presents the experimental evaluation. Section V discusses limitations and threats to validity. Finally, Section VI concludes the paper and outlines future work.

II. BACKGROUND AND RELATED WORK

Token-efficient vulnerability detection draws on bodies of work that have rarely been considered together: analysers and detectors for smart contract vulnerabilities, techniques that reduce the input an analysis component must process, and

the labelled benchmarks against which both are evaluated. This section reviews them in that order. Sections II-A to II-C cover rule-based program analysis, learning-based detection, and source-level detectors built on language models, with attention to how much input each consumes. Section II-D reviews input reduction and program slicing, Section II-E the benchmarks used for evaluation, and Section II-F positions AnchorSlice against the closest prior work, as summarised in Table 1.

A. RULE-BASED SECURITY ANALYSIS

Oyente introduced symbolic execution of EVM bytecode and defined several of the fault patterns still in use [3]. Securify derived compliance and violation patterns from dependence graphs to make its verdicts explainable [18], while SmartCheck worked on an intermediate XML representation of Solidity to keep analysis lightweight [19]. Mythril combined symbolic execution with taint analysis over bytecode [20], and MAIAN targeted trace-level properties such as contracts that can be made to leak or lock funds [21]. VeriSmart pushed precision further for arithmetic safety by inferring transaction invariants [22]. Dynamic approaches complement these: ContractFuzzer generated transaction sequences against an instrumented EVM [23], and sFuzz added an adaptive strategy for branches that random input rarely reaches [24]. Slither occupies a middle ground, converting Solidity to an SSA-based intermediate form that supports fast detectors and code-review aids [7]. The large-scale comparisons of these tools [5] and the injection studies that probe them [6] both conclude that coverage remains partial and precision low, which is the gap learning-based and model-based detectors have tried to close. Because rule-based tools execute fixed analyses rather than reading tokens, input length is not their bottleneck; it becomes one only for the detectors reviewed next.

B. LEARNING-BASED VULNERABILITY DETECTION

VulDeePecker showed that vulnerability detection could be learned from code gadgets, slices of semantically related statements gathered around library and API calls [25]. SySeVR generalised the idea to four syntactic seed types and used program dependence graphs for slicing [26]. Devign moved to graph representations that combine abstract syntax, control flow and data flow [27], and LineVul demonstrated that transformer models can localise predictions to individual lines [28], often building on code-specific pre-training such as CodeBERT [29]. In the contract setting, Zhuang *et al.* represented functions as normalised graphs and applied a degree-free graph convolutional network and a temporal message propagation network [8]; later work combined such graph models with expert-designed patterns to improve interpretability [30]. AnchorSlice is related to this literature mainly through the seed-and-expand idea: like VulDeePecker and SySeVR it begins from syntactic anchors, but it produces a reduced source for a general-purpose detector rather than a

feature vector for a trained classifier, and it therefore needs no dependence graph.

C. LANGUAGE-MODEL DETECTORS AND THEIR INPUT COST

Code-trained models [31], together with in-context learning [32] and chain-of-thought prompting [33], made it practical to obtain a security judgement from a general model. GPTScan uses model reasoning to nominate candidate variables and statements and dedicated static modules to validate them, reporting recall above 70% on ground-truth logic bugs [9]. GPTLens splits the task between an auditor role that proposes vulnerabilities and a critic role that scores them, which suppresses over-reporting [10]. LLM4Vuln decouples the components of vulnerability reasoning so that knowledge retrieval, tool use and reasoning can be measured separately [34]. LLM-SmartAudit organises multiple agents around a conversation about the contract [35], and SmartLLaMA post-trains an open model in two stages to produce both a decision and an explanation [36]. iAudit fine-tunes a detector and a reasoner and then has ranker and critic agents debate the cause, reaching an F1-score of 91.21% on a curated set of 263 vulnerabilities [11]. PropertyGPT goes further and generates formal properties for verification rather than verdicts alone [37]. What these detectors share is the assumption that they receive the contract, and the systematic review of the area identifies context length as a recurring constraint [12]. Our earlier framework, LineGuard [16], already narrows the detector's view: for a vulnerability category given in advance, it ranks candidate lines with category-specific regular expressions and passes the top-ranked candidates with a fixed context window. That pruning serves line-level localisation of one category per query, and its effect on evidence retention and token cost was not isolated. AnchorSlice is orthogonal to all of these detectors, since any of them could consume a sliced contract instead of a whole one.

D. INPUT REDUCTION AND PROGRAM SLICING

The cost of long inputs has been attacked generically. LLM-Lingua uses a small language model to identify and drop low-information tokens, reporting high compression with limited loss [14], and LongLLMLingua adds question-aware reordering for long-context settings [38]. Retrieval-augmented generation avoids the problem by fetching only relevant passages [39]. These methods estimate relevance statistically and therefore cannot promise that a particular security-critical line survives. Classical program slicing offers the opposite guarantee, computing the statements that can affect a criterion [15], and thin slicing restricts the result to statements that produce the value of interest [40]. Full slicing needs a compilable program and a dependence graph, which is a strong requirement for a corpus of deployed contracts spanning many compiler versions. AnchorSlice sits between these positions: it is coarser than dependence-based slicing, but its slicing criterion is the vulnerability taxonomy itself, and

TABLE 1: Positioning of AnchorSlice Against Prior Work on Vulnerability Detection and Input Reduction

Work	Year	Technique	Input to the detector	Input reduction	Model-free reduction	Taxonomy-guided
[3] Oyente	2016	Symbolic execution	EVM bytecode	None	–	–
[18] Securify	2018	Dependence-graph patterns	EVM bytecode	None	–	–
[19] SmartCheck	2018	XML-based patterns	Solidity source	None	–	–
[7] Slither	2019	SSA-based static analysis	Solidity source	None	–	–
[25] VulDeePecker	2018	Deep learning	Code gadgets	Slicing around API calls	✓	X
[26] SySeVR	2022	Deep learning	Semantic slices	Dependence-based slicing	✓	X
[8] Zhuang et al.	2020	Graph neural network	Contract graph	Graph normalisation	✓	X
[40] Thin slicing	2007	Program slicing	Program	Value-flow slicing	✓	X
[14] LLMingua	2023	Prompt compression	Compressed prompt	Small-model token dropping	X	X
[38] LongLLMLingua	2024	Prompt compression	Compressed prompt	Question-aware token dropping	X	X
[16] LineGuard	2025	Model-based line localisation	Pruned candidate lines	Per-category regex pruning	✓	Single category
AnchorSlice	2026	Anchor-guided static slicing	Tagged multi-category slice	Anchored windows, library collapse	✓	✓

its evidence retention against a labelled benchmark can be measured directly.

E. LABELLED BENCHMARKS

Evaluation in this area depends on the quality of labels. ScrawlID labelled real contracts by majority vote over five analysers [41], and DAppSCAN built large source and bytecode datasets from 1,199 audit reports covering 682 DApp projects [42]. We use DIVE [17], which annotates 22,330 contracts deployed between 2016 and 2024 against the eight DASP categories. DIVE aggregates MAIAN, Mythril, Semgrep, Slither, Solhint and VeriSmart through a power-based voting scheme and then re-checks positive findings against category-specific code evidence, a post-hoc validation that corrected about 14.3% of DoS and 24.9% of time-manipulation labels. Labels are assigned at contract level, which shapes how evidence retention can be validated.

F. POSITIONING OF ANCHORSlice

Table 1 contrasts AnchorSlice with the closest prior work along the dimensions that matter for token-efficient detection. Rule-based analysers do not reduce their input because they do not read tokens. Learning-based detectors reduce input through slicing or graph construction, but their reduction is tied to a trained representation rather than to a vulnerability taxonomy. Prompt-compression methods reduce input for token-bound models but use a model to decide what to keep and offer no guarantee for security-relevant code. LineGuard prunes with category-specific rules but for one category at a time. AnchorSlice is, to our knowledge, the first reduction stage that is simultaneously model-free, guided by the full vulnerability taxonomy, and validated for evidence retention on a labelled smart contract benchmark.

III. ANCHORSlice

AnchorSlice is built from a single artefact, the anchor set of each DASP category, and turns it into two components that a source-level detector uses together. The first is a *static slicer*, which keeps, for every category, the code regions able to host it, so that the detector reads far fewer tokens without losing the evidence it needs. The second is a set of *anchor-derived decision rules*, generated once from the same analysis and shipped alongside the slice, which state for each category

the shapes that count as an instance of it in the corpus at hand. The division of labour between them is deliberate: the slicer governs how much the detector reads and therefore what the query costs, the decision rules govern how it judges what it reads and therefore what the query is worth, and neither consumes a model call. Fig. 1 places both components in the detection pipeline, and Algorithm 1 gives the slicing procedure, which runs in three stages: comment removal, library collapse, and anchor-guided window extraction. This section first states the principles that constrain the design (Section III-A), then describes the lexical anchors and the single semantic gate retained (Section III-B), the extraction and tag-preserving merge of windows (Section III-C), library collapse and its evidence rescue rule (Section III-D), and finally how the slice is rendered and handed to the detector (Section III-E).

A. DESIGN PRINCIPLES

Three requirements shaped the design of both components. First, they must be fully static, in the sense that every decision follows from regular expressions and brace counting; a slicer that needed model judgement to decide what to keep would spend the token budget it is meant to save. Second, it must emit each region of source at most once. Running eight independent category-specific passes would be the obvious construction, but the passes overlap heavily and their union would exceed the original file, so AnchorSlice performs a single pass and attaches a set of candidate categories to each emitted region (Algorithm 1, lines 4–14). Third, the slicer must be detector-agnostic: its output is ordinary Solidity text with comment headers, so any source-level detector can consume it without modification.

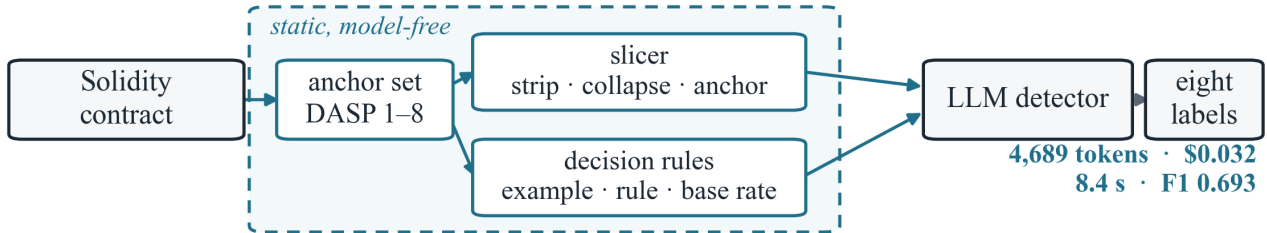
B. LEXICAL ANCHORS AND THE VISIBILITY GATE

Each category owns a set of anchor expressions, and a line matching one marks its enclosing region as a candidate for that category (line 7). Two macros carry most of the weight. `EXT` matches any external or cross-contract call, including low-level `call`, `send`, `transfer`, `delegatecall` and `staticcall` forms, calls through an interface `cast`, and calls on a receiver that is not a language builtin. `MUTFN` matches any public or external function that is not declared `view` or `pure`. The second macro deserves comment.

CONVENTIONAL USE



ANCHORSlice



The slicer sets how much the detector reads; the decision rules set how it judges what it reads.

FIGURE 1: AnchorSlice and the practice it replaces. Conventionally the whole contract is handed to the detector with a one-line list of the categories. AnchorSlice derives two static components from one anchor set per DASP category: a slicer, which emits a tagged slice of 56.5% of the source characters while retaining all 463 labelled category instances, and a set of decision rules, which state what each category means in this corpus. The detector, Claude Opus 5, and the output schema are identical in all three arms; figures beneath each lane are per contract.

Access-control faults are often an absence rather than a presence, so no positive pattern can find them; treating every state-mutating entry point as a candidate is the only static way to surface a contract whose fault is a missing modifier.

We evaluated five semantic gates intended to suppress obviously safe regions and retained one. Suppressing arithmetic candidates in contracts compiled under Solidity 0.8, where overflow reverts by default, cut arithmetic recall from 1.00 to 0.43, because DIVE labels many such contracts positive regardless. Requiring a state write before admitting a reentrancy candidate cut its recall from 0.91 to 0.36. Treating a call wrapped in `require` as a checked return cut that category from 0.60 to 0.20. Only the visibility gate survived: a `view` or `pure` body cannot mutate state, and excluding such bodies from the five state-dependent categories cost no recall at all (line 5). The rejected gates were sound as Solidity reasoning but disagreed with the benchmark’s annotation policy, which is a useful reminder that a slicer tuned to a dataset inherits that dataset’s conventions. Table 2 gives the complete anchor set, the gate applied to each category, and the evidence retention the resulting slicer achieves.

C. WINDOW EXTRACTION AND TAG-PRESERVING MERGE

An anchored line is expanded to the innermost brace block that contains it, capped at $\kappa = 10$ lines; past that cap the window falls back to the anchor with $\pi = 2$ lines of context on each side (line 9). The enclosing function signature is attached separately so that visibility and modifiers stay visible even when the body is truncated. Windows are then sorted and merged: any two whose ranges overlap or abut become a single block whose tag set is the union of theirs

(line 14). A region relevant to both reentrancy and arithmetic is consequently emitted once and carries both labels, which preserves the single-emission principle. Across the evaluation corpus this produces 16.6 blocks per contract.

D. LIBRARY COLLAPSE AND EVIDENCE RESCUE

Unmodified reference implementations account for a large share of a typical file. AnchorSlice detects top-level `SafeMath`, `Address`, `Context`, `Ownable`, `ERC20` and common exchange interfaces by name and replaces each with a one-line stub, so the detector still knows the construct is present (line 2). Applied naively, this cost recall, dropping access control to 0.95 and DoS to 0.86, because in a few contracts the only anchor for a category lay inside a collapsed construct. A rescue pass repairs the loss: if a category ends the pass with no block anywhere in the file while an anchor for it exists inside collapsed text, a three-line window is emitted at that anchor (lines 15–19). The rescue restores recall to 1.00 at a cost of about two percentage points of size.

E. SLICE RENDERING AND DETECTOR INTERFACE

The emitted slice begins with the stubs for collapsed libraries and the contract-level declarations, then lists the blocks in source order (line 20). Each block carries a header naming its original line range and its candidate categories, and line numbers refer to the original file so that a finding can be traced back. Fig. 2 shows an excerpt of the slice produced for a contract from the evaluation sample. The header reports that 61 of 128 source lines were kept in five blocks, and `SafeMath` is reduced to a stub. Block 2 is a modifier tagged for access control because it constrains `msg.sender` and

TABLE 2: Anchor Specification for the Eight DASP Categories

Category	Anchor expressions	Gate	Retained
Reentrancy	EXT	view/pure	98/98
Access Control	modifier·only\w+·(·)owner =·tx.origin·selfdestruct·suicide·delegatecall· require(msg.sender·renounce·transferOwnership·authorize·admin·MUTFN	view/pure	144/144
Arithmetic	[+ - * / %]=·++·--·x [+ * / %] y·x - y·**·<<·>>·unchecked	–	84/84
Unchecked Return Values	EXT	view/pure	38/38
DoS	for(·while(·.push(·.length·require(·revert·assert(·EXT	view/pure	26/26
Bad Randomness	blockhash· block.{difficulty, coinbase, gaslimit, number, timestamp, prevrandao}·now·keccak256·sha3(·sha256·random·nonce·seed	–	16/16
Front Running	approve(· allowance· msg.value· tx.gasprice· price· bid· swap*(· slippage· amountOutMin· deadline· reserve·MUTFN	view/pure	15/15
Time manipulation	now· block.timestamp· block.number· N {days, hours, minutes, seconds, weeks}·deadline·startTime·endTime·cooldown·lastClaim·timeout	–	42/42
All categories	–	–	463/463

EXT matches low-level calls (.call, .call.value, .send, .transfer, delegatecall, staticcall), calls through an interface cast, transferFrom/_transfer, swap and add-liquidity calls, payable (conversions, inline assembly, and calls on any receiver that is not a language builtin. MUTFN matches any public or external function not declared view or pure. The gate column names the only semantic restriction retained after ablation. *Retained* counts the label-positive contracts that keep at least one block tagged with the category.

Algorithm 1 AnchorSlice

Require: Source S , categories $\mathcal{C}$ with anchors $\{A_c\}$, gated set $\mathcal{V} \subseteq \mathcal{C}$, cap κ , padding π

Ensure: Annotated slice Σ

```

1:  $L \leftarrow \text{StripComments}(S)$ 
2:  $K \leftarrow$  lines of top-level library units in  $L$  {collapsed}
3:  $W \leftarrow \emptyset$ 
4: for each function  $f = (f_s, f_e, sig)$  in  $L$  with  $f_s \notin K$  do
5:    $G \leftarrow \mathcal{V}$  if  $sig$  is view/pure else  $\emptyset$ 
6:   for  $k = f_s$  to  $f_e$  do
7:      $T \leftarrow \{c \in \mathcal{C} \setminus G : A_c \text{ matches } L[k]\}$ 
8:     if  $T \neq \emptyset$  then
9:        $(a, b) \leftarrow \text{InnermostBlock}(L, f, k, \kappa, \pi)$ 
10:       $W \leftarrow W \cup \{(a, b, T)\}$ 
11:     end if
12:   end for
13: end for
14:  $B \leftarrow \text{MergeAdjacent}(W)$  {union of tags}
15: for each  $c \in \mathcal{C}$  tagged in no block of  $B$  do
16:   if some  $k \in K$  has  $A_c$  matching  $L[k]$  then
17:      $B \leftarrow B \cup \{(k-1, k+1, \{c\})\}$  {rescue}
18:   end if
19: end for
20:  $\Sigma \leftarrow \text{Render}(\text{stubs}(K), \text{declarations}(L), B)$ 
21: if  $|\Sigma| \geq |L|$  then
22:    $\Sigma \leftarrow L$  {small contract: send whole}
23: end if
24: return  $\Sigma$ 

```

for DoS because it contains `require`; block 3 is a public constructor-style function tagged by MUTFN for access control and front running. A short preamble tells the detector that these tags are lexical hints rather than findings and that most tagged blocks are not vulnerable. A guard handles small files: when the block headers would cost more than the removed lines save, the contract is sent whole with comments stripped (lines 21–23), which occurred for 43 of the 200 contracts in

our sample.

```

// SLICED CONTRACT 19288: 128 src lines -> 61 kept (48%),
// 5 candidate blocks.
// Each block lists the DASP categories whose static
// anchors fired inside it.
// Omitted code matched no anchor for any category.
// [standard SafeMath omitted - unmodified reference
// implementation]
// --- DECLARATIONS ---
pragma solidity ^0.4.18;
[...]
// --- BLOCK 2 | src L16-21 | candidates: Access Control,
// DoS
modifier onlyCLevel() {
  require(
    msg.sender == ceoAddress
  );
  _;
}

// --- BLOCK 3 | src L23-26 | candidates: Access Control,
// Front Running
function AdPotato() public{
  ceoAddress=msg.sender;
  initialize(0
    x58AFF91f5b48245Bd83deeB2C7d31875f68b3f0D);
}
[...]

```

FIGURE 2: Excerpt of the AnchorSlice output for DIVE contract 19288. Lines marked [. . .] are elided here for space; all other lines are reproduced verbatim.

IV. EXPERIMENTAL EVALUATION

This section evaluates what each component of AnchorSlice contributes. The detector is held fixed and three arms are compared on the same contracts: the complete source with a one-line instruction, the complete source with the anchor-derived decision rules, and the two components together. Section IV-A describes the experimental setup and configuration, Section IV-B the dataset and sampling procedure, and Section IV-C the evaluation protocol and metrics. Sections IV-D to IV-G report evidence retention, token consumption and runtime, detection effectiveness, and per-category behaviour.

TABLE 3: Experimental Configuration and Parameters

Parameter	Value	Description
<i>Slicer</i>		
Window cap κ	10 lines	Largest brace block kept whole
Padding π	± 2 lines	Context around an anchor past the cap
Gated categories $\mathcal{V}$	5	Skipped inside <code>view/pure</code> bodies
<i>Sampling</i>		
Sample size	200	Seeded stratified sample, three rounds
Category floor	5	Minimum positives and negatives per category
Size filter	1.2–30 kB	Excludes empty stubs, bounds per-call cost
<i>Detector</i>		
Model	Opus 5	Claude Opus 5, high reasoning effort
Access	Claude CLI	Print mode, tools and tool servers disabled
Output	JSON schema	8 binary labels, confidences, reasons
Instruction	per arm	One-line category list (A) or anchor-derived decision rules (B, C)
Calls	1 per contract	Sequential, 900 s timeout
Sampling	not exposed	Temperature and top-p not user-controllable

Section IV-H draws the three findings together.

A. SETUP

AnchorSlice and the evaluation harness are implemented in Python using only the standard library, and the complete source, sample definitions, raw results and run logs are available in the project repository [43]. The exact slicer used for the reported runs is archived there unchanged, alongside a later revision that adds guards for contract shapes this sample does not contain. Slicer runtime was measured on an Intel Core i9-9880H processor with Python 3.12.3. The detector in every arm is Claude Opus 5 at high reasoning effort, accessed through the Claude Code command-line interface in non-interactive print mode. All built-in tools and external tool servers are disabled and no session state is persisted, so the detector sees nothing beyond the prompt and cannot retrieve the answer from elsewhere. A JSON schema constrains every reply to eight binary labels with a confidence and a short justification per category, which removes output parsing as a source of error. Each contract receives a single call, calls are issued sequentially, and the interface reports token usage, latency and cost for every call. The interface does not expose sampling parameters such as temperature, so the results are reproducible in configuration and in the logged per-call record but not bit-for-bit. Table 3 lists every parameter used; all of them are held fixed across the three arms.

B. DATASET AND SAMPLING

We drew 200 contracts from DIVE [17] with seeded random sampling, repaired so that every category carries enough pos-

itive and negative instances to be measurable in both directions; a sample composed only of heavily labelled contracts would leave precision undefined, because a detector answering all-positive would score perfectly. Contracts smaller than 1,200 bytes were excluded as empty stubs, and contracts larger than 30,000 bytes were excluded to bound the cost of each call. The sample totals 2,311,110 characters and no contract required truncation. The 1,600 label cells (200 contracts $\times$ 8 categories) contain 463 positives, a positive rate of 28.9%. Per-category rates track the corpus closely for the common categories – access control 72.0% against 74.9% corpus-wide, reentrancy 49.0% against 51.1% and arithmetic 42.0% against 42.7% – while the two rarest categories are deliberately over-sampled, bad randomness at 8.0% against 2.8% and front running at 7.5% against 2.7%, so that they carry enough positives to be scored at all. The sample was drawn in three rounds; the 130 contracts of the last round were never inspected while the method was developed and are reported separately in Section IV-F.

C. EVALUATION PROTOCOL

The evaluation uses three arms over the same contracts, so that each component of AnchorSlice can be credited separately. Arm A is how a source-level detector is used in practice: the complete contract with a short instruction naming the eight categories, one line each, and asking for a fault where it is reachable and exploitable. Arm B keeps the complete contract but replaces that instruction with the anchor-derived decision rules, and therefore isolates the second component. Arm C is AnchorSlice as proposed: the slice, preceded by a short preamble explaining the block format and stating that the category tags are lexical hints rather than findings, together with the same decision rules, and therefore adds the first component on top of Arm B. The detector, reasoning effort, output schema and contracts are identical throughout, with one call per contract. Because the three arms classify the same 1,600 cells, every comparison is paired, and we report McNemar’s exact test on discordant cells alongside the aggregate measures.

We report label agreement over all cells, Hamming loss, exact match at contract level, and micro-averaged precision, recall and F1-score. Because the label distribution is unbalanced, we also report Cohen’s kappa, which corrects for agreement expected by chance. Resource use is measured as total tokens across all token classes, wall-clock time, and cost. Evidence retention is measured without the detector, as the share of label-positive contracts that keep at least one block tagged with the corresponding category. All detection metrics use the DIVE labels as the reference.

D. EVIDENCE RETENTION AND SLICE COMPACTNESS

AnchorSlice reduced the sample from 2,311,110 to 1,305,860 characters, or 56.5% of the original. Evidence retention was 1.00 for every category: all 463 label-positive category instances kept at least one block tagged with that category, as summarised in Table 2. Reaching full retention required

TABLE 4: Resource Consumption Across 200 Contracts

Measure	Arm A	Arm B	Arm C	C vs B
Input tokens	1,837,975	2,222,346	825,285	−62.9%
Output tokens	254,939	247,767	112,469	−54.6%
Contract text cached	1,100,024	1,072,714	339,432	−68.4%
Total tokens	2,092,914	2,470,113	937,754	−62.0%
Tokens per contract	10,465	12,351	4,689	−62.0%
Wall clock (s)	4,151	5,363	1,673	−68.8%
Seconds per contract	20.8	26.8	8.4	−68.8%
Cost (USD)	17.75	17.52	6.45	−63.2%
Cost per contract	\$0.089	\$0.088	\$0.032	−63.2%

four guards that a smaller sample had not exercised: legacy contracts in which a function without a visibility keyword is public by default, function signatures spanning several lines, anonymous fallback functions, and minified sources in which an entire contract occupies one or two lines. A contract for which no candidate block can be isolated, or whose block headers would cost more than the removed lines save, is sent whole with comments stripped, which occurred for 43 of the 200 contracts. Decomposing the reduction by stage, comment removal alone brings the sample to 77.8% of its original size and library collapse and anchoring take it to 56.5%, so removal accounts for most of the compression while the anchoring stage returns part of its saving in block headers. The slicer processes a contract in 7.6 ms on average and 17.5 ms at most, which is negligible beside the tens of seconds a detector call takes.

E. TOKEN CONSUMPTION, RUNTIME AND COST

Table 4 and Fig. 3 report resource use. The decision rules are not free: stating them costs roughly two thousand tokens per call, so Arm B reads 12,351 tokens per contract where Arm A reads 10,465, and takes 26.8 seconds against 20.8. The slicer more than repays that. Arm C consumes 4,689 tokens per contract, 62.0% fewer than Arm B and 55.2% fewer than Arm A, at \$0.032 against \$0.088 and \$0.089, and 8.4 seconds against 26.8 and 20.8. The contract text the detector must read, measured as the tokens written to the prompt cache, falls by 68.4% between Arm B and Arm C. Output length falls with it, by 54.6%: given a compact, tagged input the detector writes shorter justifications, so the saving compounds on the side that is billed most heavily. Over the whole sample this is \$6.45 against \$17.52 for the same decision rules on complete sources.

F. DETECTION EFFECTIVENESS

Table 5 separates the two components. Replacing the one-line instruction with the anchor-derived decision rules, while still submitting the complete contract, moves every aggregate measure at once: recall rises from 0.322 to 0.801, precision from 0.365 to 0.614, F1-score from 0.342 to 0.695, Cohen’s kappa from 0.097 to 0.547 and agreement from 0.642 to 0.797. Missed vulnerabilities fall from 314 to 92 while false positives fall from 259 to 233, so both error types improve

TABLE 5: Detection Effectiveness Over 1,600 Label Cells

Metric	Arm A	Arm B	Arm C
Label agreement	0.642	0.797	0.801
Exact match (8/8)	5/200	25/200	28/200
Precision	0.365	0.614	0.627
Recall	0.322	0.801	0.775
F1-score	0.342	0.695	0.693
Cohen’s kappa	0.097	0.547	0.549
True positives	149	371	359
False positives	259	233	214
False negatives	314	92	104

Arm A: complete source, one-line instruction. Arm B: complete source, anchor-derived decision rules. Arm C: AnchorSlice, slice and decision rules. A vs B $p \approx 7.1 \times 10^{-24}$; B vs C $p = 0.57$; A vs C $p \approx 3.7 \times 10^{-26}$.

together and the movement is not a shift of the decision threshold. Over the 1,600 paired cells, 185 are correct only in Arm A and 433 only in Arm B, a two-sided McNemar exact p of about 7.1×10^{-24} . The detection quality of the method is therefore attributable to its second component.

Adding the slicer leaves that quality intact. Arm C reaches an F1-score of 0.693 against 0.695, an agreement of 0.801 against 0.797 and a kappa of 0.549 against 0.547, and the paired test finds no difference: 53 cells are correct only with the complete source and 60 only with the slice, $p = 0.57$. It trades twelve recovered vulnerabilities for nineteen suppressed false positives, and raises the number of contracts labelled correctly in all eight categories from 25 to 28. Read together with Table 4, this is the central result of the paper: removing 43.5% of the source characters, and with them 62.0% of the tokens, two thirds of the latency and 63.2% of the cost, costs nothing in detection quality that the experiment can measure.

Against ordinary practice, the method as a whole raises F1-score from 0.342 to 0.693 and cuts missed vulnerabilities from 314 to 104 while spending 55.2% fewer tokens ($p \approx 3.7 \times 10^{-26}$). The 130 contracts of the final sampling round, which were never inspected while the anchors and the decision rules were developed, give the same picture on their own: an F1-score of 0.689 and an agreement of 0.797, against 0.340 and 0.634 for ordinary practice, so the result is not an artefact of development on the evaluation set.

G. PER-CATEGORY BEHAVIOUR

Table 6 and Fig. 4 report every category for the three arms. The decision rules account for the movement, and it is concentrated where the one-line instruction is most conservative. Reentrancy rises from an F1-score of 0.19 to 0.82 and access control from 0.41 to 0.82: asked only for faults that are reachable and exploitable, the detector marked 10 of 98 reentrancy contracts and 42 of 144 access-control contracts, whereas a rule that states what the corpus counts as an instance, supported by the MUTFN anchor over state-mutating entry points, recovers most of them. Unchecked return values rise from 0.46 to 0.80 and DoS from 0.17 to 0.42.

Adding the slice leaves most categories within a point

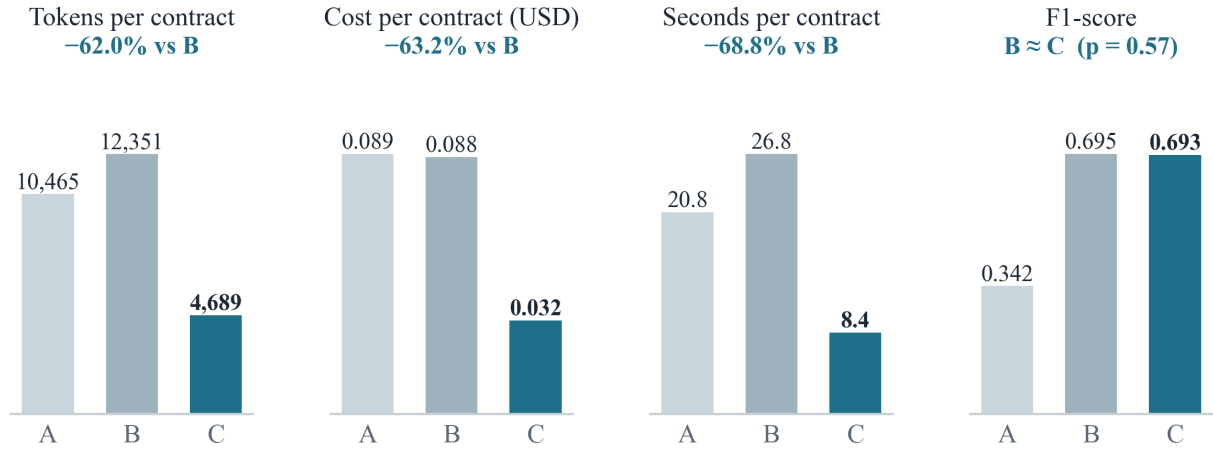


FIGURE 3: Resource use and detection quality per contract across the three arms. The decision rules raise both token use and latency; the slicer removes far more than they add, at the same detection quality.

or two of Arm B and redistributes a little error: DoS and bad randomness gain, arithmetic and unchecked return values lose, and reentrancy, access control and time manipulation are unchanged in substance. Two categories resist both components. Time manipulation falls from 0.67 in Arm A to about 0.59, because the rule recovers misses, from 15 to 4, while raising false positives from 11 to roughly 50. Front running falls to zero in both Arm B and Arm C: the rule suppresses the ERC-20 approve pattern that dominates the corpus, cutting 132 false positives to 16 and 14 respectively, but recovers none of its 15 positives. That category is produced by a single analyser in the reference and carries no lexical signal a detector can act on, as Section V discusses.

H. DISCUSSION

Three findings follow from Tables 4 and 5, one for each thing the experiment varies.

The first concerns what the detector is told. A one-line definition per category leaves a capable model markedly conservative: asked to report faults that are reachable and exploitable, it recovers under a third of the labelled vulnerabilities. Stating, per category, which shapes the corpus counts as an instance nearly doubles the F1-score without a change of model, of schema, or of the code shown. Much of the headroom usually attributed to model capability is therefore headroom in the instruction.

The second concerns what the detector reads. Removing 43.5% of the source characters left every aggregate measure unchanged, and the paired test cannot distinguish the sliced arm from the one that reads the complete source. The simplest reading is that the removed text was never used: comment blocks, licence headers and unmodified library implementations carry no information on which a security judgement depends. This is consistent with the observation that evidence buried inside long inputs is attended to less reliably than evidence near the edges [13]: a shorter input raises the density of

security-relevant material and shortens the distance between related fragments. The slicer therefore buys efficiency rather than accuracy, which is what a preprocessing stage should do.

The third concerns how the two combine. The decision rules cost about two thousand tokens per call, so on complete sources they make the query more expensive, 12,351 tokens against 10,465, and slower, 26.8 seconds against 20.8. The slicer returns that outlay several times over, ending at 4,689 tokens and 8.4 seconds, so the pair is cheaper than the practice it replaces as well as more accurate. Neither half achieves this alone: rules without a slice leave the token cost untouched, and a slice without rules leaves the detector guessing what the reference counts as a vulnerability.

The stage decomposition adds a final observation about the anchors themselves. Most of the compression comes from removal, and the anchoring stage returns part of its saving in block headers, so the value of anchoring lies less in compression than in localisation. An anchor is then useful in proportion to how selective it is: the two categories whose tags fire on roughly one block in eight, bad randomness and time manipulation, are the two the one-line instruction already handled best, whereas the categories whose tags fire on more than half of all blocks are those where the decision rule, rather than the tag, carries the improvement.

V. LIMITATIONS AND THREATS TO VALIDITY

The results above come from one detector, one benchmark and a single run per arm, and several properties of the design and of the evaluation bound how far they generalise. This section states these limits explicitly so that the reported numbers are interpreted within them. It first sets out what the three-arm design does and does not separate and why the monetary saving travels less well than the token count, then the two categories the per-category analysis shows the method fails to handle, and finally the threats arising from how evidence retention was validated, from the sample and the number of runs, and from the single detector used.

TABLE 6: Per-Category Detection Detail Across All Eight DASP Categories

Category	DIVE +	F1 (A)	F1 (B)	F1 (C)	FN (A)	FN (B)	FN (C)
Reentrancy	98	0.19	0.82	0.82	88	1	2
Access Control	144	0.41	0.82	0.83	102	12	9
Arithmetic	84	0.44	0.57	0.52	53	28	38
Unchecked Return Values	38	0.46	0.80	0.74	20	10	13
DoS	26	0.17	0.42	0.47	19	16	15
Bad Randomness	16	0.58	0.58	0.62	9	9	8
Front Running	15	0.09	0.00	0.00	8	15	15
Time manipulation	42	0.67	0.60	0.58	15	1	4
All categories	463	0.342	0.695	0.693	314	92	104

DIVE + counts reference positives; FN counts missed vulnerabilities. Arm A: complete source with the one-line instruction. Arm B: complete source with the anchor-derived decision rules. Arm C: AnchorSlice, slice and decision rules together.

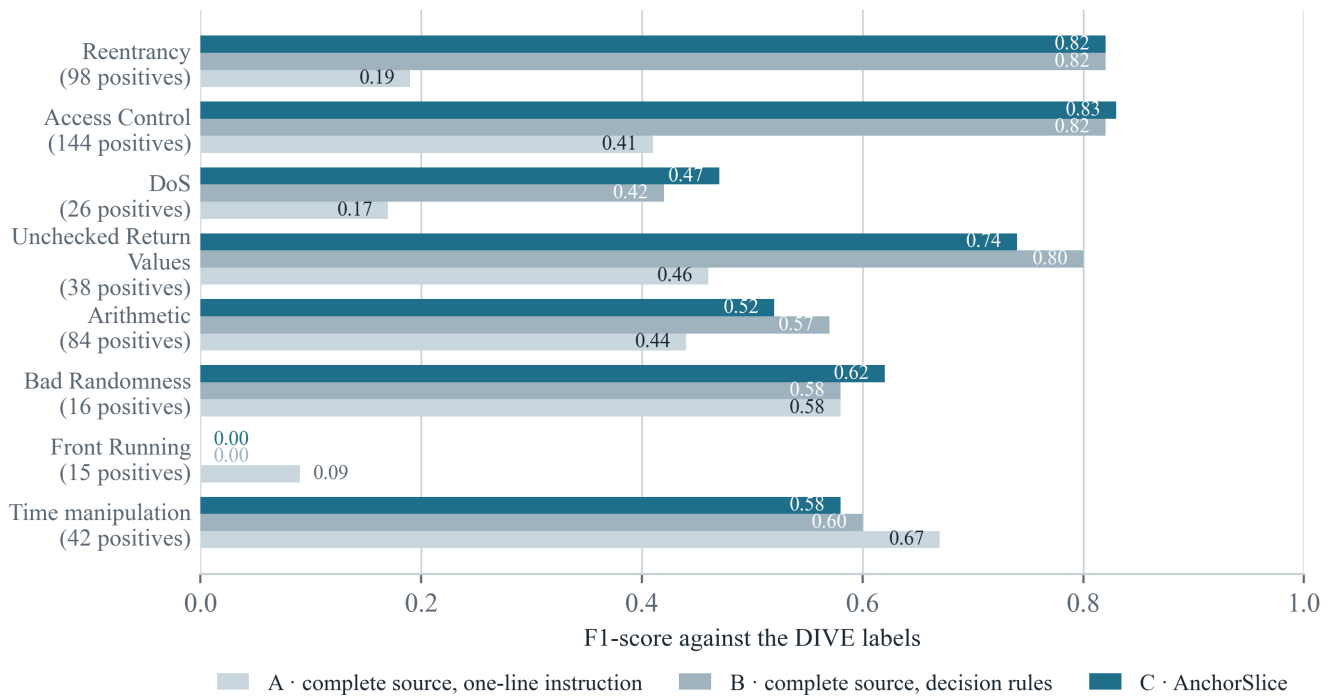


FIGURE 4: F1-score per DASP category for the three arms, ordered by the size of the change between Arm A and Arm C. The decision rules account for nearly all of the movement; adding the slice leaves each category within a few points of Arm B.

What the three arms do and do not separate. The design isolates each component once: Arm B against Arm A measures the decision rules on complete sources, and Arm C against Arm B measures the slicer under fixed rules. It does not separate the two effects the slice itself combines, namely removing code and naming candidate categories in the block headers; a tag-free slice would be needed for that, and the opposite movements of DoS and unchecked return values suggest that these two effects can act against each other, so the aggregate comparison may understate the benefit of removal alone. Nor does it establish that the components behave independently on other corpora, since both derive from the same anchor set and are calibrated against the same reference.

Portability of the cost figure. Cost fell slightly faster than token count because the removed tokens are input rather

than output. Detectors whose pricing weights these classes differently, or that bill cached input at a different rate, will see a different monetary figure, so token count and latency are the more portable benefits.

Two categories the method does not solve. Front running is the clearest failure: once the decision rules are in force the detector marks none of its 15 positives, trading the baseline's 132 false positives for 16 in Arm B and 14 in Arm C at the cost of every true detection. In the reference, that category is produced by a single analyser, and no lexical pattern we measured predicts it: the ERC-20 approve allowance race, the textbook shape, appears in 164 of the 200 contracts and predicts the label with a precision of 0.04. Time manipulation moves the other way, cutting missed vulnerabilities from 15 to 4 at the cost of raising false positives from 11 to 50. Relatedly,

library collapse identifies constructs by name, so a contract that reuses a standard name for a modified implementation would have that code collapsed as well.

Weak retention criterion. DIVE labels contracts rather than lines, so evidence retention of 1.00 means only that a positive contract keeps some block tagged with that category, not that it keeps the specific statements an annotator had in mind. The DIVE labels are themselves derived from a consensus of static analysers with post-hoc validation, so effectiveness is measured against the benchmark's annotation policy rather than against audited ground truth.

Development set and run-to-run variation. The anchors and the calibrated decision rules were developed against a subset of the corpus, so figures over the whole sample are in part an in-sample fit; we therefore also report the 130 contracts of the final sampling round, which were never inspected during development and on which the result is unchanged. Each arm was run once. Three repetitions of one configuration on a fixed subset varied by 4.2 percentage points of agreement, which bounds the noise in single-run comparisons and is an order of magnitude smaller than the effect reported here.

The nature of the reference. DIVE's labels are produced by six analysers that are standard practice in this field – MAIAN, Mythril, Semgrep, Slither, Solhint and VeriSmart – combined by power-based voting and then re-checked against category-specific code evidence, a validation that corrected about 14.3% of DoS and 24.9% of time-manipulation labels [17]. It is therefore a carefully constructed reference rather than an unfiltered tool dump, and reproducing it closely is a meaningful result; it is nonetheless a tool consensus rather than an audited ground truth, so agreement with it measures reproduction of what these analysers collectively find, and our decision rules follow that consensus where it departs from Solidity semantics. A benchmark with human-audited labels would test the same method against a different notion of correctness.

One detector. All three arms use a single proprietary model accessed through an interface that does not expose sampling parameters or allow a dated model snapshot to be pinned. Although both components are detector-agnostic by construction, their effect on other source-level detectors, including open-weight models and multi-stage pipelines such as LineGuard [16], has not been measured.

VI. CONCLUSION AND FUTURE WORK

This paper addressed the token cost of smart contract vulnerability detection and showed that a large share of that cost can be removed before the detector runs. We presented AnchorSlice, a static method that reduces a Solidity contract to the fragments capable of hosting each DASP category and tells the detector what each category means there, and evaluated both of its components on the DIVE benchmark. This section summarises the findings and the directions they open.

AnchorSlice is lexical throughout, needs no compilation, and processes a contract in 7.6 ms on average; both of its components are derived from one anchor set and neither costs a model call. On 200 contracts it retained evidence for all 463 labelled category instances, a retention of 1.00 in every category, while emitting 56.5% of the original characters. Three arms over the same contracts credit each component separately. The decision rules raise the F1-score from 0.342 to 0.695 on complete sources, cutting missed vulnerabilities from 314 to 92 ($p \approx 7.1 \times 10^{-24}$), and the slicer then removes 62.0% of the tokens, 63.2% of the cost and 68.8% of the latency with no detectable change in detection quality ($p = 0.57$). Against ordinary practice the method doubles the F1-score while spending 55.2% fewer tokens, and the 130 contracts never used in development give the same result. Because both components are static, they compose at no inference cost: a practitioner who today pastes a contract into a detector can adopt AnchorSlice without changing the detector, the schema, or anything else in the pipeline.

Four directions follow. A tag-free arm would separate the effect of removing code from the effect of naming candidate categories, and would also remove the header overhead that currently offsets the anchoring stage. A selectivity threshold that suppresses any category tagging more than a set fraction of a contract's blocks would address the DoS failure directly while preserving the access-control gain. The anchors should be re-validated on a held-out DIVE split, with the protocol repeated across several runs, before the approach is applied to the full corpus. Finally, the slice should be evaluated in front of other detectors, including open-weight models and our LineGuard framework, to establish that the token savings transfer across detection architectures.

VII. DISCLOSURE STATEMENT

The authors declare that they have no relevant or material financial interests related to the research described in this paper.

REFERENCES

- [1] Szabo, N.: 'Formalizing and Securing Relationships on Public Networks', *First Monday*, 1997, 2, (9), doi: [10.5210/fm.v2i9.548](https://doi.org/10.5210/fm.v2i9.548)
- [2] Wood, G.: 'Ethereum: A Secure Decentralised Generalised Transaction Ledger', *Ethereum Project Yellow Paper*, 2014, <https://ethereum.github.io/yellowpaper/paper.pdf>
- [3] Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A.: 'Making Smart Contracts Smarter'. *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2016, pp. 254–269, doi: [10.1145/2976749.2978309](https://doi.org/10.1145/2976749.2978309)
- [4] NCC Group: 'Decentralized Application Security Project (DASP) Top 10', 2018, <https://dasp.co>
- [5] Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: 'Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts'. *Proc. ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020, pp. 530–541, doi: [10.1145/3377811.3380364](https://doi.org/10.1145/3377811.3380364)
- [6] Ghaleb, A., Pattabiraman, K.: 'How Effective Are Smart Contract Analysis Tools? Evaluating Smart Contract Static Analysis Tools Using Bug Injection'. *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2020, pp. 415–427, doi: [10.1145/3395363.3397385](https://doi.org/10.1145/3395363.3397385)
- [7] Feist, J., Grieco, G., Groce, A.: 'Slither: A Static Analysis Framework for Smart Contracts'. *Proc. IEEE/ACM Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15, doi: [10.1109/WETSEB.2019.00008](https://doi.org/10.1109/WETSEB.2019.00008)

- [8] Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q.: 'Smart Contract Vulnerability Detection Using Graph Neural Network'. *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2020, pp. 3283–3290, doi: [10.24963/ijcai.2020/454](https://doi.org/10.24963/ijcai.2020/454)
- [9] Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y.: 'GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis'. *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2024, pp. 1–13, doi: [10.1145/3597503.3639117](https://doi.org/10.1145/3597503.3639117)
- [10] Hu, S., Huang, T., İlhan, F., Tekin, S.F., Liu, L.: 'Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives'. *Proc. IEEE Int. Conf. Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 2023, pp. 297–306, doi: [10.1109/TPS-ISA58951.2023.00044](https://doi.org/10.1109/TPS-ISA58951.2023.00044)
- [11] Ma, W., Wu, D., Sun, Y., Wang, T., Liu, S., Zhang, J., Xue, Y., Liu, Y.: 'Combining Fine-Tuning and LLM-Based Agents for Intuitive Smart Contract Auditing with Justifications'. *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2025, pp. 1–13, arXiv:2403.16073
- [12] He, Z., Li, Z., Yang, S., Qiao, A., Zhang, X., Luo, X., Chen, T.: 'Large Language Models for Blockchain Security: A Systematic Literature Review', *arXiv preprint*, 2024, arXiv:2403.14280
- [13] Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: 'Lost in the Middle: How Language Models Use Long Contexts', *Trans. Assoc. Computational Linguistics*, 2024, 12, pp. 157–173, doi: [10.1162/tac1_a_00638](https://doi.org/10.1162/tac1_a_00638)
- [14] Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., Qiu, L.: 'LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models'. *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 13358–13376, doi: [10.18653/v1/2023.emnlp-main.825](https://doi.org/10.18653/v1/2023.emnlp-main.825)
- [15] Weiser, M.: 'Program Slicing'. *Proc. Int. Conf. Software Engineering (ICSE)*, 1981, pp. 439–449, <https://dl.acm.org/doi/10.5555/800078.802557>
- [16] Pakshad, P., Aghanasiri, S., Shameli-Sendi, A., Omar, M.: 'LineGuard: An LLM-Based Smart Contract Vulnerability Detection', manuscript under review, 2025, <https://github.com/sjag77/LineGuard>
- [17] Alsunaidi, S.J., Aljamaan, H., Hammoudeh, M.: 'DIVE: A Multi-Label Smart Contract Vulnerability Dataset', *Scientific Data*, 2026, 13, art. 664; dataset: [10.5281/zenodo.18519253](https://doi.org/10.5281/zenodo.18519253)
- [18] Tsankov, P., Dan, A., Drachler-Cohen, D., Gervais, A., Bünzli, F., Vechev, M.: 'Securify: Practical Security Analysis of Smart Contracts'. *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018, pp. 67–82, doi: [10.1145/3243734.3243780](https://doi.org/10.1145/3243734.3243780)
- [19] Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y.: 'SmartCheck: Static Analysis of Ethereum Smart Contracts'. *Proc. IEEE/ACM Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2018, pp. 9–16, doi: [10.1145/3194113.3194115](https://doi.org/10.1145/3194113.3194115)
- [20] ConsenSys: 'Mythril: Security Analysis Tool for EVM Bytecode', 2018, <https://github.com/ConsenSys/mythril>
- [21] Nikolić, I., Kolluri, A., Sergey, I., Saxena, P., Hobor, A.: 'Finding the Greedy, Prodigal, and Suicidal Contracts at Scale'. *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2018, pp. 653–663, doi: [10.1145/3274694.3274743](https://doi.org/10.1145/3274694.3274743)
- [22] So, S., Lee, M., Park, J., Lee, H., Oh, H.: 'VeriSmart: A Highly Precise Safety Verifier for Ethereum Smart Contracts'. *Proc. IEEE Symp. Security and Privacy (S&P)*, 2020, pp. 1678–1694, doi: [10.1109/SP40000.2020.00032](https://doi.org/10.1109/SP40000.2020.00032)
- [23] Jiang, B., Liu, Y., Chan, W.K.: 'ContractFuzzer: Fuzzing Smart Contracts for Vulnerability Detection'. *Proc. ACM/IEEE Int. Conf. Automated Software Engineering (ASE)*, 2018, pp. 259–269, doi: [10.1145/3238147.3238177](https://doi.org/10.1145/3238147.3238177)
- [24] Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: 'sFuzz: An Efficient Adaptive Fuzzer for Solidity Smart Contracts'. *Proc. ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020, pp. 778–788, doi: [10.1145/3377811.3380334](https://doi.org/10.1145/3377811.3380334)
- [25] Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: 'VulDeePecker: A Deep Learning-Based System for Vulnerability Detection'. *Proc. Network and Distributed System Security Symp. (NDSS)*, 2018, doi: [10.14722/ndss.2018.23158](https://doi.org/10.14722/ndss.2018.23158)
- [26] Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: 'SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities', *IEEE Trans. Dependable and Secure Computing*, 2022, 19, (4), pp. 2244–2258, doi: [10.1109/TDSC.2021.3051525](https://doi.org/10.1109/TDSC.2021.3051525)
- [27] Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: 'Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks'. *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 10197–10207, arXiv:1909.03496
- [28] Fu, M., Tantithamthavorn, C.: 'LineVul: A Transformer-Based Line-Level Vulnerability Prediction'. *Proc. IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, 2022, pp. 608–620, doi: [10.1145/3524842.3528452](https://doi.org/10.1145/3524842.3528452)
- [29] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M.: 'CodeBERT: A Pre-Trained Model for Programming and Natural Languages'. *Findings of the Association for Computational Linguistics: EMNLP*, 2020, pp. 1536–1547, doi: [10.18653/v1/2020.findings-emnlp.139](https://doi.org/10.18653/v1/2020.findings-emnlp.139)
- [30] Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X.: 'Combining Graph Neural Networks with Expert Knowledge for Smart Contract Vulnerability Detection'. *IEEE Trans. Knowledge and Data Engineering*, 2023, 35, (2), pp. 1296–1310, doi: [10.1109/TKDE.2021.3095196](https://doi.org/10.1109/TKDE.2021.3095196)
- [31] Chen, M., et al.: 'Evaluating Large Language Models Trained on Code', *arXiv preprint*, 2021, arXiv:2107.03374
- [32] Brown, T.B., et al.: 'Language Models Are Few-Shot Learners'. *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901, arXiv:2005.14165
- [33] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D.: 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models'. *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 24824–24837, arXiv:2201.11903
- [34] Sun, Y., Wu, D., Xue, Y., Liu, H., Ma, W., Zhang, L., Shi, M., Liu, Y.: 'LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning', *arXiv preprint*, 2024, arXiv:2401.16185
- [35] Wei, Z., Sun, J., Zhang, Z., Zhang, X., Yang, X., Zhu, L.: 'LLM-SmartAudit: Advanced Smart Contract Vulnerability Detection', *arXiv preprint*, 2024, arXiv:2410.09381
- [36] Yu, L., Chen, S., Yuan, H., Wang, P., Huang, Z., Zhang, J., Shen, C., Zhang, F., Yang, J., Li, J.: 'Smart-LLaMA: Two-Stage Post-Training of Large Language Models for Smart Contract Vulnerability Detection and Explanation', *arXiv preprint*, 2024, arXiv:2411.06221
- [37] Liu, Y., Xue, Y., Wu, D., Sun, Y., Li, Y., Shi, M., Liu, Y.: 'PropertyGPT: LLM-Driven Formal Verification of Smart Contracts Through Retrieval-Augmented Property Generation'. *Proc. Network and Distributed System Security Symp. (NDSS)*, 2025, doi: [10.14722/ndss.2025.241357](https://doi.org/10.14722/ndss.2025.241357)
- [38] Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y., Qiu, L.: 'LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression'. *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 1658–1677, doi: [10.18653/v1/2024.acl-long.91](https://doi.org/10.18653/v1/2024.acl-long.91)
- [39] Lewis, P., et al.: 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks'. *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474, arXiv:2005.11401
- [40] Sridharan, M., Fink, S.J., Bodík, R.: 'Thin Slicing'. *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2007, pp. 112–122, doi: [10.1145/1250734.1250748](https://doi.org/10.1145/1250734.1250748)
- [41] Yashavant, C.S., Kumar, S., Karkare, A.: 'ScrawlID: A Dataset of Real World Ethereum Smart Contracts Labelled with Vulnerabilities', *arXiv preprint*, 2022, arXiv:2202.11409
- [42] Zheng, Z., Su, J., Chen, J., Lo, D., Zhong, Z., Ye, M.: 'DAppSCAN: Building Large-Scale Datasets for Smart Contract Weaknesses in DApp Projects', *IEEE Trans. Software Engineering*, 2024, 50, (6), pp. 1360–1373, doi: [10.1109/TSE.2024.3383422](https://doi.org/10.1109/TSE.2024.3383422)
- [43] Repository available at <https://github.com/sjag77/AnchorSlice>



SAJAD AGHANASIRI received his M.Sc. in Software Engineering from Shahid Beheshti University (SBU), Tehran, Iran, in 2026, and his B.Sc. in Computer Engineering from Islamic Azad University, Central Tehran Branch (IAU-CTB). His thesis addressed securing smart contracts through vulnerability detection in code snippets and program analysis, exploring static, semantic, and hybrid techniques. His research interests span software security, blockchain/smart-contract security, program analysis, and complex networks. He has served as a Teaching Assistant for one academic term.



SINA ZAKER is currently an M.Sc. student in Network Security at Shahid Beheshti University (SBU), Tehran, Iran. He received his B.Sc. in Computer Engineering. His thesis focuses on defending Graph Neural Networks (GNNs) against structural adversarial attacks, exploring robust mitigation strategies and adaptive spectral filtering models. His primary research interests span network security, artificial intelligence security, Web Application Firewalls (WAF), complex graphs, adversarial machine learning, and the security of distributed cloud systems.



ALIREZA SHAMELI-SENDI is currently an Associate Professor at Shahid Beheshti University. Before joining SBU, he was a Postdoctoral Fellow at Ericsson, Canada and Postdoctoral at ETS and McGill universities in collaboration with Ericsson. He received his Ph.D degree in computer engineering from Montreal University (Ecole Polytechnique de Montreal), Canada. He obtained his B.Sc. and M.Sc. from Amirkabir University of Technology. His primary research interests include information security, intrusion response system, and cloud computing. He is a recipient of Postdoctoral Research Fellowship Award and Industrial Postdoctoral Fellowship Award from Canada. In addition, he received the best researcher awards, at SBU, in 2018 and 2022.